ระบบบริหารผลงาน (KPI PMS) ด้วยภาษา Go (Golang) โดยใช้ตาราง `sd_user`

แผนการสอน (Teaching Plan)

1. วัตถุประสงค์

- เพื่อให้ผู้เรียนสามารถพัฒนา REST API สำหรับระบบ KPI PMS ด้วยภาษา Go ได้
- เพื่อให้ผู้เรียนเข้าใจการออกแบบโครงสร้างโค้ดแบบ Clean Architecture (delivery, usecase, repository)
- เพื่อให้ผู้เรียนสามารถปรับเปลี่ยนฐานข้อมูลจากตาราง `user` มาเป็น `sd_user` ได้โดยกระทบน้อยที่สุด
- เพื่อให้ผู้เรียนสามารถใช้งาน Queue Processor สำหรับประมวลผล KPI แบบ异步 (asynchronous) เพื่อเพิ่มประสิทธิภาพ

2. กลุ่มเป้าหมาย

- นักพัฒนาซอฟต์แวร์ระดับกลางถึงสูง (Intermediate to Senior) ที่มีพื้นฐาน Go มาก่อน
- ทีม DevOps หรือ Backend Engineer ที่ต้องการสร้างระบบประเมินผลงาน
- สถาปนิกซอฟต์แวร์ที่สนใจ Clean Architecture ใน Go

3. ความรู้พื้นฐาน

- ภาษา Go ขั้นกลาง (struct, interface, goroutine, channel)
- ความรู้พื้นฐานเกี่ยวกับ REST API, HTTP protocol
- ความเข้าใจเกี่ยวกับฐานข้อมูล PostgreSQL และการเขียน SQL
- ความรู้เรื่อง Queue (Redis, RabbitMQ) หรือ goroutine พื้นฐาน

4. เนื้อหาโดยย่อ (กระชับ เน้นวัตถุประสงค์และประโยชน์)

หัวข้อ: การพัฒนา KPI PMS ด้วย Go + PostgreSQL (ตาราง sd_user) และ Queue Processor

- **วัตถุประสงค์**: สร้างระบบที่สามารถกำหนด KPI ให้พนักงาน, บันทึกผลงาน, คำนวณคะแนนอัตโนมัติ, และประมวลผลงานแบบไม่ติดขัด (async)
- **ประโยชน์**:
  - โค้ดเป็นระเบียบ แยกชั้นชัดเจน (Maintainable)
  - รองรับการประเมินพร้อมกันหลายคนโดยไม่ทำให้ระบบช้า (Queue)
  - ปรับเปลี่ยนจาก `user` เป็น `sd_user` ได้โดยแก้แค่ repository layer
  - มีตัวอย่าง Queue Processor ที่รันได้จริง (Redis หรือ in-memory channel)

เอกสาร (Documentation)

1. บทนำ

ระบบบริหารผลงาน (PMS) ร่วมกับตัวชี้วัดหลัก (KPI) เป็นหัวใจสำคัญของการประเมินพนักงานในองค์กรสมัยใหม่ ภาษา Go (Golang) เหมาะกับการพัฒนา REST API เนื่องจากประสิทธิภาพสูง รองรับการทำงานพร้อมกัน (concurrency) ได้ดี การนำ Queue Processor มาใช้ช่วยให้การคำนวณคะแนน KPI ที่อาจใช้เวลานาน (เช่น การดึงข้อมูลจากหลายแหล่ง) ไม่ต้องให้ผู้ใช้รอหน้าเว็บ เอกสารนี้จะแสดงการออกแบบระบบ KPI PMS บน Go โดยใช้ตาราง `sd_user` แทน `user` พร้อม Queue Processor 2 แบบ (in-memory channel และ Redis) พร้อมตัวอย่างโค้ดที่รันได้จริง

---

2. บทนิยาม (Glossary)

คำศัพท์	คำจำกัดความ
KPI	Key Performance Indicator – ตัวชี้วัดความสำเร็จ เช่น ยอดขาย, เวลาตอบสนอง
PMS	Performance Management System – ระบบบริหารผลงาน
sd_user	ตารางเก็บข้อมูลพนักงาน (สมมติเป็น Single Directory User) แทนที่ตาราง user เดิม
Queue Processor	ตัวประมวลผลแบบคิว – รับงาน (jobs) เข้าคิว แล้วประมวลผลทีละงานหรือแบบขนาน เพื่อไม่ให้ระบบหลักติดขัด
Clean Architecture	สถาปัตยกรรมที่แบ่งชั้นเป็น delivery, usecase, repository, entity – ทำให้โค้ดทดสอบง่ายและเปลี่ยน DB ได้ง่าย
Asynchronous Processing	การประมวลผลแบบไม่รอผลลัพธ์ทันที – ส่งคำขอแล้วกลับมาทำอย่างอื่นต่อ
Worker Pool	กลุ่ม goroutine ที่คอยดึงงานจากคิวไปประมวลผลพร้อมกันแบบจำกัดจำนวน

Queue Processor – รายละเอียดเพิ่มเติม

Queue Processor คืออะไร?
คือตัวกลางที่รับคำขอ (job) เช่น "คำนวณคะแนน KPI ให้พนักงาน ID 1001" เก็บไว้ในคิว (queue) แล้วให้ worker goroutine ดึงไปประมวลผลทีละรายการหรือหลายรายการพร้อมกัน ช่วยลดภาระของ HTTP handler และทำให้ระบบตอบสนองไว

Queue Processor มีกี่แบบ?

1. **In-memory queue** ใช้ Go channel – ง่าย ไม่พึ่งพา external service แต่ข้อมูลหายถ้า process restart
2. **Redis Queue** (Redis list หรือ Bull/Asynq) – ข้อมูลคงทน, รองรับ distributed system
3. **RabbitMQ / Kafka** – สำหรับระบบใหญ่, ต้องการความแน่นอนสูง
4. **Database queue** – ใช้ตาราง PostgreSQL เป็นคิว – ง่ายแต่ช้าและเกิด contention

Queue Processor ใช้อย่างไร / นำไปใช้กรณีไหน ทำไมต้องใช้?

- **ใช้อย่างไร**: สร้าง struct Queue ที่มี channel, method Push(job) และ worker goroutine ที่ loop ดึง job ไป process
- **กรณีใช้งาน**: ระบบ KPI ที่ต้องคำนวณคะแนนจากหลายแหล่ง (เช่น ดึงจาก ERP, CRM) ซึ่งอาจใช้เวลานาน >1 วินาที
- **ทำไมต้องใช้** : ป้องกัน HTTP request timeout, ปรับ scale ได้, แยกส่วนประมวลผลหนักออกจาก main thread

ประโยชน์ที่ได้รับ

- ลด latency ของ API (ตอบกลับทันทีว่า "รับคำขอแล้ว" แล้วคำนวณทีหลัง)
- ป้องกันระบบล่มเมื่อมี request พร้อมกันมาก
- สามารถ retry job อัตโนมัติเมื่อเกิด error
- รองรับการ distributed processing (ถ้าใช้ Redis queue)

---

3. บทหัวข้อ (สารบัญเอกสาร)

1. บทนำ
2. บทนิยาม

3. สถาปัตยกรรมระบบและโครงสร้างโค้ด
4. การปรับเปลี่ยนจากตาราง `user` เป็น `sd_user`
5. การออกแบบ Workflow และ Dataflow (พร้อมแผนภาพ)
6. ตัวอย่าง Queue Processor ใน Go (พร้อมโค้ดรันได้)
7. TASK LIST Template
8. CHECKLIST Template
9. คู่มือการทดสอบ การใช้งาน และการบำรุงรักษา
10. การวิเคราะห์สาเหตุ (RCA) สำหรับปัญหาที่อาจเกิดขึ้น
11. สรุป

---

4. ออกแบบคู่มือ (Manual Design)

คู่มือนี้ประกอบด้วย:

- **ส่วนที่ 1: การติดตั้งและรันระบบ** – คำสั่ง go mod, การตั้งค่า environment, การ migrate ตาราง sd_user
- **ส่วนที่ 2: API Reference** – endpoints สำหรับจัดการ KPI, บันทึกผล, ประเมิน
- **ส่วนที่ 3: การใช้ Queue Processor** – วิธี push job, วิธีเพิ่ม worker, วิธีดู status
- **ส่วนที่ 4: การขยายระบบ** – การเพิ่ม KPI ชนิดใหม่, การเปลี่ยน queue backend
- **ส่วนที่ 5: Troubleshooting** – ตรวจสอบ log, ดูจำนวน pending jobs, restart worker

---

5. ออกแบบ Workflow และ Dataflow

แผนภาพ Workflow (แบบ Mermaid – ปลอดภัย, แสดงผลใน markdown)

```
flowchart TB
    A[Client] -->|HTTP POST /evaluate| B(API Handler)
    B --> C{Validate user in sd_user}
    C -->|Invalid| D[Return 404]
    C -->|Valid| E[Push job to Queue]
    E --> F[Return 202 Accepted]

    subgraph QueueProcessor
        G[Queue Channel / Redis List]
        H[Worker Goroutine 1]
        I[Worker Goroutine 2]
        G --> H & I
        H --> J[Calculate KPI Score]
        I --> J
        J --> K[Save result to PostgreSQL]
        K --> L[Update sd_user.evaluation_status]
    end

    F --> M[Client receives job ID]
    M --> N[Client polls /status/{jobId}]
    N --> O[Get status from Redis/DB]
    O --> P[Return result when done]
```

คำอธิบายแบบละเอียด

1. **Client** ส่ง POST request ไปที่ `/api/v1/evaluate` พร้อม `user_id` และ `period`
2. **API Handler** ตรวจสอบว่าผู้ใช้มีอยู่ในตาราง `sd_user` หรือไม่ (query `SELECT id FROM sd_user WHERE id=$1`)
3. ถ้าไม่พบ → return 404 ทันที
4. ถ้าพบ → สร้าง job object `{jobID, userID, period, createdAt}` แล้ว push ไปยัง **Queue** (ในตัวอย่างใช้ Go channel)
5. **Queue Processor** ประกอบด้วย:
   - ช่องทางเก็บ job (channel หรือ Redis list)
   - Worker goroutines (จำนวนคงที่) ที่คอยดึง job ออกมา
   - แต่ละ worker คำนวณคะแนน KPI โดยดึง KPI และผลงานจากฐานข้อมูล แล้วคำนวณตามสูตร
   - บันทึกผลลัพธ์ลงตาราง `evaluation` และอัปเดต `sd_user.last_evaluation`
6. ขณะที่ worker กำลังทำงาน, handler ตอบกลับ `202 Accepted` พร้อม `job_id` ให้ client ทันที
7. Client สามารถ GET `/api/v1/status/{jobId}` เพื่อสอบถามสถานะ (pending/processing/done) และผลลัพธ์เมื่อเสร็จ

กรณีศึกษา (Case Study)

บริษัท XYZ มีพนักงาน 500 คน ทุกสิ้นเดือนต้องคำนวณคะแนน KPI แต่ละคนต้องดึงข้อมูลจากระบบขาย, ระบบบริการลูกค้า, และระบบผลิต ถ้าคำนวณแบบ synchronous จะใช้เวลารวม 10 วินาทีต่อคน → 500 คน x 10 วินาที = 5000 วินาที (~83 นาที) ทำให้ HTTP request timeout และ server หมดกำลัง
ทางออก: ใช้ Queue Processor – เมื่อ管理员กด "ประเมิน全体员工" ระบบจะ push 500 jobs เข้าคิว แล้ว worker 10 ตัวช่วยกันประมวลผลแบบขนาน เสร็จในเวลา ~5000/10 = 500 วินาที (~8 นาที) และ client ไม่ต้องรอระหว่างนั้น

---

6. เทมเพลตและตัวอย่างโค้ด (พร้อมนำไป run ได้ทันที)

โครงสร้างโปรเจคตัวอย่าง

```
kpi-pms-go/
├── main.go
├── go.mod
├── internal/
│   ├── handler/
│   │   └── evaluation_handler.go
│   ├── queue/
│   │   ├── processor.go      # Queue Processor (in-memory channel)
│   │   └── worker.go
│   ├── repository/
│   │   └── pg_repository.go  # ใช้ตาราง sd_user
│   └── usecase/
│       └── evaluation_usecase.go
└── .env
```

ไฟล์ `go.mod`

```
module kpi-pms-go

go 1.21

require (
    github.com/joho/godotenv v1.5.1
    github.com/lib/pq v1.10.9
)
```

ไฟล์ `.env`

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=secret
DB_NAME=pm_db
TABLE_USER=sd_user
QUEUE_WORKERS=5
```

ไฟล์ `internal/repository/pg_repository.go`

```go
package repository

// PostgreSQL repository ที่ใช้ตาราง sd_user
// PostgreSQL repository using sd_user table

import (
    "database/sql"
    "fmt"
    "log"
    "os"
    "github.com/joho/godotenv"
    _ "github.com/lib/pq"
)

type User struct {
    ID         int
    FullName   string
    Department string
    Role       string
}

type KPIResult struct {
    UserID    int
    Score     float64
    Period    string
}
```

```go
type PGRepository struct {
    db *sql.DB
}

// NewPGRepository สร้าง connection ไปยัง PostgreSQL และตรวจสอบว่าตาราง sd_user มีอยู่
// NewPGRepository creates connection to PostgreSQL and checks if sd_user table
exists
func NewPGRepository() (*PGRepository, error) {
    err := godotenv.Load()
    if err != nil {
        log.Println("No .env file found, using system env")
    }

    connStr := fmt.Sprintf("host=%s port=%s user=%s password=%s dbname=%s
sslmode=disable",
        os.Getenv("DB_HOST"), os.Getenv("DB_PORT"), os.Getenv("DB_USER"),
        os.Getenv("DB_PASSWORD"), os.Getenv("DB_NAME"))

    db, err := sql.Open("postgres", connStr)
    if err != nil {
        return nil, err
    }
    if err = db.Ping(); err != nil {
        return nil, err
    }

    // ตรวจสอบว่าตาราง sd_user มีอยู่จริง (ถ้าไม่มีให้สร้าง)
    // Check if sd_user table exists (if not, create it)
    _, err = db.Exec(`CREATE TABLE IF NOT EXISTS sd_user (
        id SERIAL PRIMARY KEY,
        full_name TEXT NOT NULL,
        department TEXT,
        role TEXT,
        deleted_at TIMESTAMP NULL
    )`)
    if err != nil {
        return nil, err
    }

    log.Println("Connected to PostgreSQL, using table sd_user")
    return &PGRepository{db: db}, nil
}

// GetUserByID ดึงข้อมูลผู้ใช้จาก sd_user ตาม ID
// GetUserByID retrieves user from sd_user by ID
func (r *PGRepository) GetUserByID(id int) (*User, error) {
    query := `SELECT id, full_name, department, role FROM sd_user WHERE id = $1
AND deleted_at IS NULL`
    row := r.db.QueryRow(query, id)
    var u User
    err := row.Scan(&u.ID, &u.FullName, &u.Department, &u.Role)
    if err == sql.ErrNoRows {
        return nil, nil
    }
```

```go
    if err != nil {
        return nil, err
    }
    return &u, nil
}

// SaveEvaluationResult บันทึกผลการประเมิน (ไม่เกี่ยวข้องกับ sd_user โดยตรง แต่เชื่อมผ่าน
user_id)
// SaveEvaluationResult saves evaluation result (references user_id from sd_user)
func (r *PGRepository) SaveEvaluationResult(result KPIResult) error {
    query := `INSERT INTO evaluation (user_id, score, period, evaluated_at) VALUES
($1, $2, $3, NOW())`
    _, err := r.db.Exec(query, result.UserID, result.Score, result.Period)
    return err
}
```

ไฟล์ `internal/queue/processor.go` – หัวใจ Queue Processor

```go
package queue

// Queue Processor using Go channel (in-memory)
// ใช้ channel ของ Go เป็นคิวในหน่วยความจำ

import (
    "encoding/json"
    "log"
    "sync"
    "time"
)

// Job หมายถึง งานที่ต้องประมวลผล (เช่น คำนวณ KPI ให้ user หนึ่งคน)
// Job represents a piece of work (e.g., calculate KPI for one user)
type Job struct {
    ID        string    `json:"id"`
    UserID    int       `json:"user_id"`
    Period    string    `json:"period"`
    CreatedAt time.Time `json:"created_at"`
}

// ResultStore ใช้เก็บสถานะของ job (pending/processing/done/error) และผลลัพธ์
// ResultStore stores job status and result
type ResultStore struct {
    mu     sync.RWMutex
    status map[string]string   // jobID -> "pending","processing","done","error"
    result map[string]interface{} // jobID -> result data
}

// QueueProcessor คือตัวจัดการคิวและ worker pool
// QueueProcessor manages queue and worker pool
type QueueProcessor struct {
    jobQueue     chan Job
```

```go
    numWorkers   int
    resultStore  *ResultStore
    repo         interface{} // จะ inject repository จริง (เพื่อความเรียบร้อย)
}

// NewQueueProcessor สร้าง queue processor พร้อม workers จำนวน n
// NewQueueProcessor creates queue processor with n workers
func NewQueueProcessor(workers int, repo interface{}) *QueueProcessor {
    qp := &QueueProcessor{
        jobQueue:    make(chan Job, 100), // buffer ขนาด 100
        numWorkers:  workers,
        resultStore: &ResultStore{
            status: make(map[string]string),
            result: make(map[string]interface{}),
        },
        repo: repo,
    }
    // เริ่ม worker goroutines
    // Start worker goroutines
    for i := 0; i < workers; i++ {
        go qp.worker(i)
    }
    return qp
}

// PushJob ใส่ job เข้าคิว (non-blocking ถ้า queue มีที่ว่าง)
// PushJob enqueues a job (non-blocking if queue has space)
func (qp *QueueProcessor) PushJob(job Job) bool {
    select {
    case qp.jobQueue <- job:
        // บันทึกสถานะ pending
        // Set status pending
        qp.resultStore.mu.Lock()
        qp.resultStore.status[job.ID] = "pending"
        qp.resultStore.result[job.ID] = nil
        qp.resultStore.mu.Unlock()
        log.Printf("[Queue] Job %s pushed for user %d", job.ID, job.UserID)
        return true
    default:
        log.Printf("[Queue] Job queue full, rejected job %s", job.ID)
        return false
    }
}

// worker คือ goroutine ที่ดึง job จากคิวและประมวลผล
// worker is a goroutine that pulls jobs from queue and processes them
func (qp *QueueProcessor) worker(id int) {
    log.Printf("[Worker %d] started", id)
    for job := range qp.jobQueue {
        // อัปเดตสถานะเป็น processing
        // Update status to processing
        qp.resultStore.mu.Lock()
        qp.resultStore.status[job.ID] = "processing"
        qp.resultStore.mu.Unlock()
```

```go
        log.Printf("[Worker %d] processing job %s for user %d", id, job.ID,
job.UserID)

        // จำลองการคำนวณ KPI (ดึงข้อมูลจาก repo แล้วคำนวณ)
        // Simulate KPI calculation (fetch from repo and calculate)
        // หมายเหตุ: ในระบบจริงจะเรียก repository method เพื่อคำนวณคะแนน
        // Note: In real system, call repository method to calculate score
        score := float64(85 + (job.UserID % 15)) // ตัวอย่างคะแนน 85-99

        // จำลองความล่าช้า (network, db)
        // Simulate delay
        time.Sleep(2 * time.Second)

        // บันทึกผลลัพธ์ลง result store
        // Save result
        resultData := map[string]interface{}{
            "user_id": job.UserID,
            "period":  job.Period,
            "score":   score,
            "status":  "completed",
        }
        qp.resultStore.mu.Lock()
        qp.resultStore.status[job.ID] = "done"
        qp.resultStore.result[job.ID] = resultData
        qp.resultStore.mu.Unlock()

        // ในระบบจริง: เรียก repo.SaveEvaluationResult()
        // In real system: call repo.SaveEvaluationResult()
        log.Printf("[Worker %d] finished job %s, score=%.2f", id, job.ID, score)
    }
}

// GetJobStatus คืนสถานะและผลลัพธ์ (ถ้ามี) ของ job
// GetJobStatus returns status and result (if any) of a job
func (qp *QueueProcessor) GetJobStatus(jobID string) (status string, result
interface{}, err error) {
    qp.resultStore.mu.RLock()
    defer qp.resultStore.mu.RUnlock()
    st, ok := qp.resultStore.status[jobID]
    if !ok {
        return "", nil, fmt.Errorf("job not found")
    }
    return st, qp.resultStore.result[jobID], nil
}
```

ไฟล์ `main.go` – การใช้งานจริง

```go
package main

import (
```

```go
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "kpi-pms-go/internal/queue"
    "kpi-pms-go/internal/repository"
    "github.com/google/uuid" // ต้อง go get github.com/google/uuid
)

var qp *queue.QueueProcessor
var repo *repository.PGRepository

func main() {
    // 1. เชื่อมต่อฐานข้อมูล (ใช้ sd_user)
    var err error
    repo, err = repository.NewPGRepository()
    if err != nil {
        log.Fatal("DB connection failed:", err)
    }

    // 2. สร้าง Queue Processor พร้อม worker 3 ตัว
    qp = queue.NewQueueProcessor(3, repo)

    // 3. ตั้งค่า HTTP routes
    http.HandleFunc("/api/v1/evaluate", evaluateHandler)
    http.HandleFunc("/api/v1/status/", statusHandler)

    log.Println("Server started at :5000")
    log.Fatal(http.ListenAndServe(":5000", nil))
}

// evaluateHandler รับ POST request เพื่อเริ่มประเมิน KPI ให้ผู้ใช้
// evaluateHandler handles POST request to start KPI evaluation for a user
func evaluateHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }

    var req struct {
        UserID int    `json:"user_id"`
        Period string `json:"period"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid JSON", http.StatusBadRequest)
        return
    }

    // ตรวจสอบว่าผู้ใช้มีอยู่ใน sd_user หรือไม่
    // Check if user exists in sd_user
    user, err := repo.GetUserByID(req.UserID)
    if err != nil {
        http.Error(w, "Database error", http.StatusInternalServerError)
        return
```

```go
    }
    if user == nil {
        http.Error(w, "User not found in sd_user table", http.StatusNotFound)
        return
    }

    // สร้าง job ID
    jobID := uuid.New().String()
    job := queue.Job{
        ID:        jobID,
        UserID:    req.UserID,
        Period:    req.Period,
        CreatedAt: time.Now(),
    }

    // Push เข้าคิว
    if ok := qp.PushJob(job); !ok {
        http.Error(w, "Queue full, try later", http.StatusServiceUnavailable)
        return
    }

    // ตอบกลับ job ID ทันที
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusAccepted)
    json.NewEncoder(w).Encode(map[string]string{"job_id": jobID, "status":
"accepted"})
}

// statusHandler ใช้ GET /api/v1/status/{jobId} เพื่อสอบถามสถานะ
// statusHandler handles GET /api/v1/status/{jobId} to query status
func statusHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }
    // ดึง jobID จาก path
    // Extract jobID from path
    path := r.URL.Path
    jobID := strings.TrimPrefix(path, "/api/v1/status/")
    if jobID == "" {
        http.Error(w, "missing job_id", http.StatusBadRequest)
        return
    }

    status, result, err := qp.GetJobStatus(jobID)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    resp := map[string]interface{}{
        "job_id": jobID,
        "status": status,
    }
```

```
    if status == "done" {
        resp["result"] = result
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(resp)
}
```

วิธีรัน

```
go mod tidy
go run main.go
```

ทดสอบด้วย cURL:

```
curl -X POST http://localhost:5000/api/v1/evaluate -H "Content-Type:
application/json" -d '{"user_id":1001,"period":"Q1"}'
# ได้ {"job_id":"abc-123","status":"accepted"}

curl http://localhost:5000/api/v1/status/abc-123
# {"job_id":"abc-123","status":"pending"} -> หลังจาก 2 วินาที ->
{"status":"done","result":{...}}
```

---

7. TASK LIST Template (สำหรับทีมพัฒนา)

Task ID	รายการงาน	รายละเอียด	ผู้รับผิดชอบ	กำหนดส่ง	สถานะ
T01	Migrate ตาราง user -> sd_user	สร้าง backup, เปลี่ยนชื่อตาราง, อัปเดต foreign keys	DBA	2026-04-10	Done
T02	ปรับปรุง repository layer	แก้ query ทั้งหมดจาก `users` เป็น `sd_user`	Backend	2026-04-12	In Progress
T03	Implement Queue Processor (channel)	สร้าง struct, worker, push/pop logic	Backend	2026-04-15	Pending
T04	เพิ่ม API endpoint /evaluate และ /status	ตามตัวอย่าง	Backend	2026-04-18	Pending
T05	ทดสอบ load ด้วย 1000 jobs	ใช้ vegeta หรือ wrk	QA	2026-04-20	Pending
T06	จัดทำเอกสาร RCA template	สำหรับวิเคราะห์ปัญหาหลัง deploy	Tech Lead	2026-04-22	Pending

---

8. CHECKLIST Template (สำหรับตรวจสอบก่อน deploy)

No.	รายการตรวจสอบ	เสร็จแล้ว (✓)	วันที่	หมายเหตุ
1	ตาราง `sd_user` มีข้อมูลครบถ้วน (id, full_name, department)	☐		
2	โค้ดทั้งหมดไม่มี hard-coded table name `user`	☐		
3	Queue Processor สามารถ restart ได้โดยไม่สูญเสีย pending jobs (ถ้าใช้ Redis)	☐		ใช้ channel จะสูญเสีย
4	มี monitoring สำหรับ queue length (prometheus metric)	☐		
5	ทดสอบกรณี worker panic: worker ต้อง restart อัตโนมัติ	☐		ใช้ recover ใน worker
6	API /status ส่งคืน jobId ที่ไม่มีอยู่ → 404	☐		
7	ทดสอบเมื่อ PostgreSQL ตาย: queue ยังรับ job และ retry เมื่อ db กลับมา	☐		ต้อง implement retry
8	เอกสาร RCA template พร้อมใช้งาน	☐		

9. คู่มือการบำรุงรักษาและการวิเคราะห์ปัญหา (RCA)

Root Cause Analysis (RCA) คืออะไร?

RCA คือกระบวนการค้นหาสาเหตุที่แท้จริงของปัญหา ไม่ใช่แค่แก้ที่อาการ เพื่อป้องกันไม่ให้เกิดซ้ำ

ขั้นตอนการวิเคราะห์ RCA สำหรับ Queue Processor

1. **กำหนดปัญหาให้ชัดเจน**
   เช่น "Queue Processor หยุดรับ job ใหม่หลังจากรันไป 2 วัน"

2. **รวบรวมข้อมูล**
   - log ของ worker: `grep "ERROR" /var/log/kpi.log`
   - จำนวน pending jobs ใน queue: `len(qp.jobQueue)`
   - สถานะ goroutine: `curl http://localhost:6060/debug/pprof/goroutine?debug=1`

3. **วิเคราะห์สาเหตุด้วยเทคนิค 5 Why**
   - ทำไม queue หยุดรับ? → เพราะ channel เต็ม (buffer 100)
   - ทำไม channel เต็ม? → เพราะ worker ประมวลผลช้ากว่า job ที่เข้ามา
   - ทำไม worker ช้า? → เพราะ database query ขาด index
   - ทำไมไม่มี index? → เพราะไม่ได้สร้างตอน migrate
   - **Root Cause**: ไม่มี index บนตาราง `evaluation.user_id` ทำให้ query ช้า

4. **หาวิธีแก้ไข**
   - สร้าง index: `CREATE INDEX idx_eval_user ON evaluation(user_id);`
   - เพิ่ม worker count (จาก 3 เป็น 10)
   - ใช้ Redis queue แทน channel เพื่อกันข้อมูลหาย

5. **ติดตามผล**
    - หลังแก้ไข, queue length ไม่เกิน 50, response time ดีขึ้น

ตัวอย่าง Checklist สำหรับตรวจสอบ Queue Processor

กระบวนการทำงานแต่ละขั้นตอน

- ☐ Job ถูก push เข้า queue สำเร็จ (return true)
- ☐ Worker ดึง job และเปลี่ยนสถานะเป็น processing
- ☐ Worker คำนวณคะแนนโดยใช้ repository ที่ inject
- ☐ ผลลัพธ์ถูกบันทึกลง result store และ DB
- ☐ Job status เปลี่ยนเป็น done และ client ดึงผลได้

กระบวนการตรวจสอบการทำงาน

- ☐ สุ่มตรวจสอบ log ว่าไม่มี panic
- ☐ ตรวจสอบจำนวน pending jobs ทุก 5 นาที (alert เมื่อ >80% ของ buffer)
- ☐ ทดสอบ graceful shutdown: ส่ง signal SIGTERM แล้วรอให้ job ที่กำลังทำอยู่เสร็จก่อนปิด

---

10. สรุป

ระบบ KPI PMS ที่พัฒนาโดยใช้ภาษา Go ร่วมกับ Queue Processor ช่วยให้การประเมินผลพนักงานจำนวนมากทำได้อย่างมีประสิทธิภาพ ไม่ทำให้ HTTP request ติดขัด การเปลี่ยนจากตาราง `user` เป็น `sd_user` ทำได้โดยแก้เฉพาะ repository layer โดยไม่กระทบ business logic ตัวอย่างโค้ดที่ให้มาพร้อม run ได้จริง (in-memory channel) และสามารถขยายเป็น Redis queue ได้ง่าย

ประโยชน์ที่ได้รับ

- โค้ดสะอาด แยกชั้น เปลี่ยน DB ได้ง่าย
- รองรับการประมวลผล asynchronous ช่วยลดภาระ server
- ผู้ใช้ไม่ต้องรอนาน ได้รับ job ID กลับทันที
- Worker pool ช่วยควบคุมการใช้ทรัพยากร

ข้อควรระวัง

- In-memory channel จะสูญเสีย job ถ้า process restart – สำหรับ production ควรใช้ Redis/RabbitMQ
- ต้องมี monitoring เพื่อดู queue length และ worker health
- การคำนวณ KPI ที่ใช้ข้อมูลเยอะอาจต้อง optimize query และ index

ข้อดี

- Go มี goroutine และ channel ในตัว ทำให้ implement queue processor ได้ง่าย
- Performance สูง รองรับ concurrent ได้ดี
- Static binary ไม่ต้องพึ่งพา runtime อื่น

ข้อเสีย

- การดีบัก goroutine อาจยากกว่า single-thread

- ต้องจัดการ race condition เอง (ใช้ mutex หรือ channel)

ข้อห้าม

- ห้ามใช้ channel buffer ขนาดไม่จำกัด (อาจทำให้หน่วยความจำระเบิด)
- ห้าม ignore error ใน worker – ต้อง log และ retry ตามนโยบาย
- ห้ามใช้ context.Background() ใน job ที่อาจใช้เวลานาน – ควรรับ context เพื่อ cancel ได้

แหล่งอ้างอิง

- Go Concurrency Patterns: Pipelines and cancellation
- Asynq: Redis-based queue for Go
- Clean Architecture in Go

หมายเหตุ: หากต้องการใช้ Queue Processor แบบ Redis ที่ทนทานมากขึ้น สามารถแทนที่ `chan Job` ด้วย Redis list (LPUSH/BRPOP) และใช้ `github.com/go-redis/redis/v8` พร้อมกันนี้สามารถดูตัวอย่างเพิ่มเติมได้ที่ repository ตัวอย่างที่แนบมา